Robot Dynamics & Control

2026

Session 1 – Tools for Simulation and Control

Tutors:
Pedro Luis Figueroa Saire
Danial Sabzevari
Emails: name.surname@edu.unige.it (or Teams)

Dibris

UNIVERSITÀ DEGLI STUDI
DI GENOVA

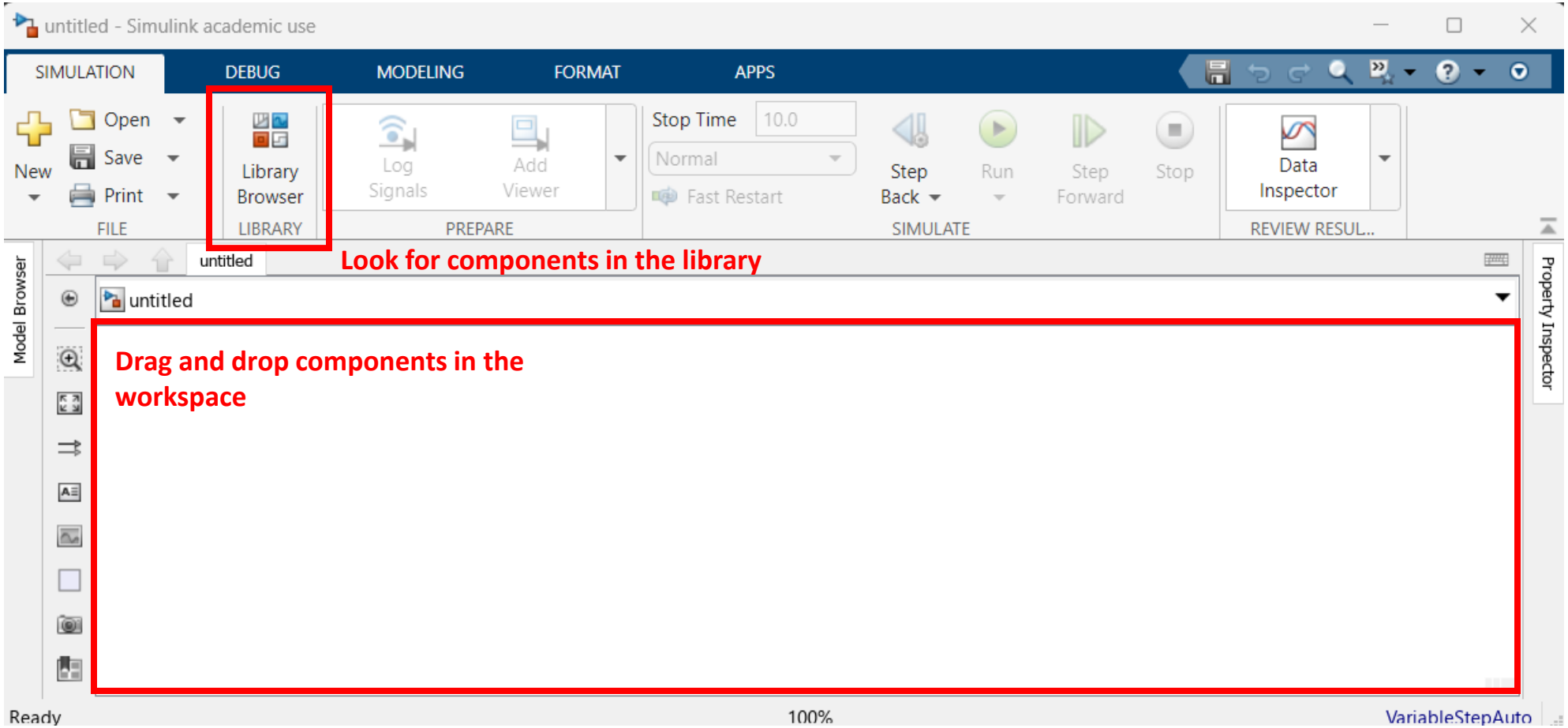
Sessions scheduling and exam

4/5 exercise sessions:

- Matlab/Simulink for **simulation and control**
- **Multibody** simulation
- **Gravity Compensation** algorithms
- **Recursive Inverse Dynamics** for trajectory following
- **Dynamic control** of manipulators
- **Environment interaction**, impedance control

Simulink for Dynamic Simulation

Used to ***find solutions*** of ***differential equations*** in **time domain**



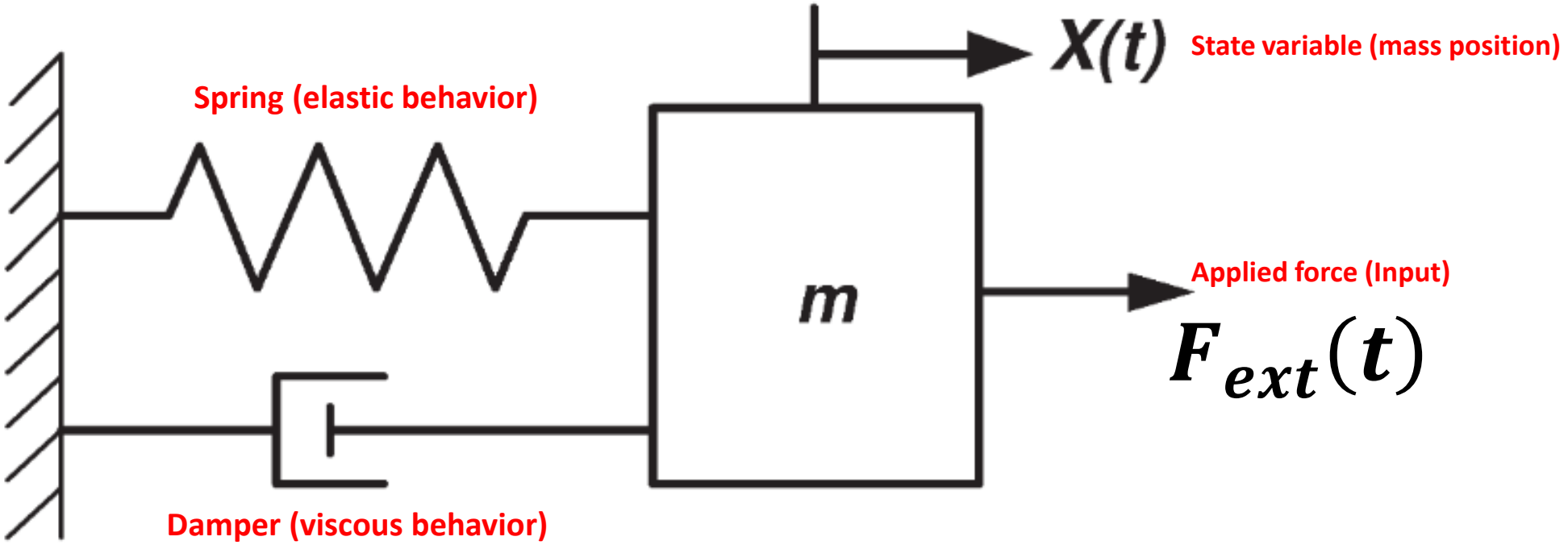
Any ***dynamical system*** (a **robot**, a motor, a circuit) can be **modeled** by **differential equations**

UNIVERSITÀ DEGLI STUDI DI GENOVA

The Mass-Spring-Damper system

Widely used model in control engineering, it's a ***second order system***, so it ***can represent the acceleration of a rigid body.***

Newton's **II** ***Law relates acceleration*** of a body ***with the force which caused it to accelerate*** >>>>>>> **DYNAMIC MODEL**



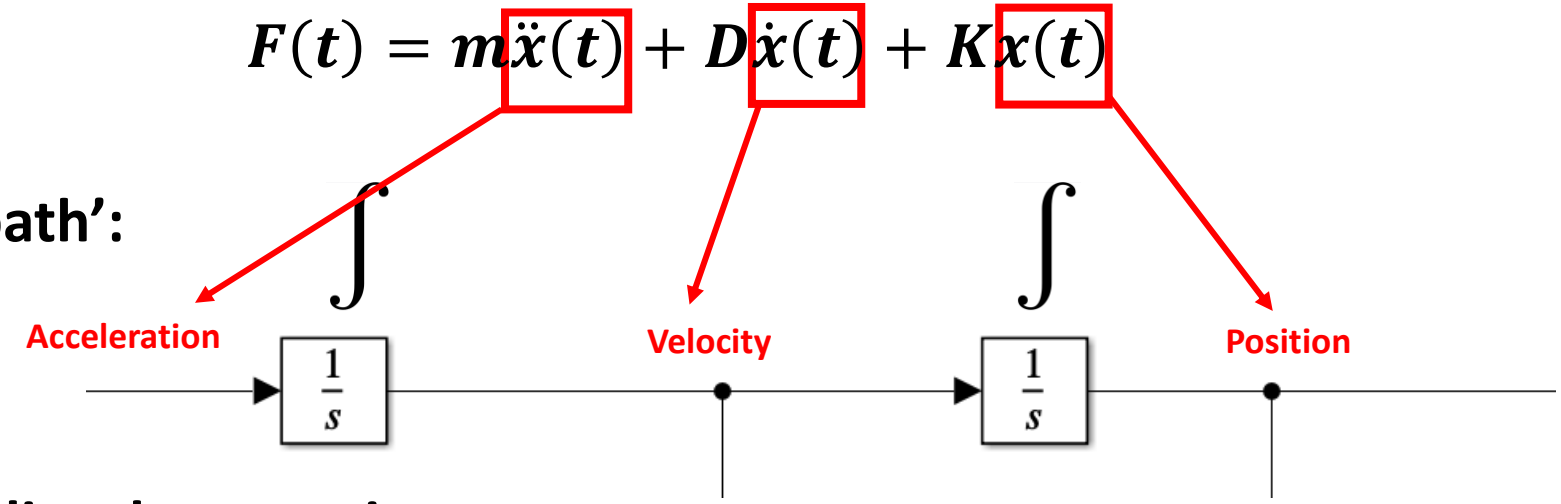
UNIVERSITÀ DEGLI STUDI
DI GENOVA

Simulate Differential Equations

From the 2° Law:

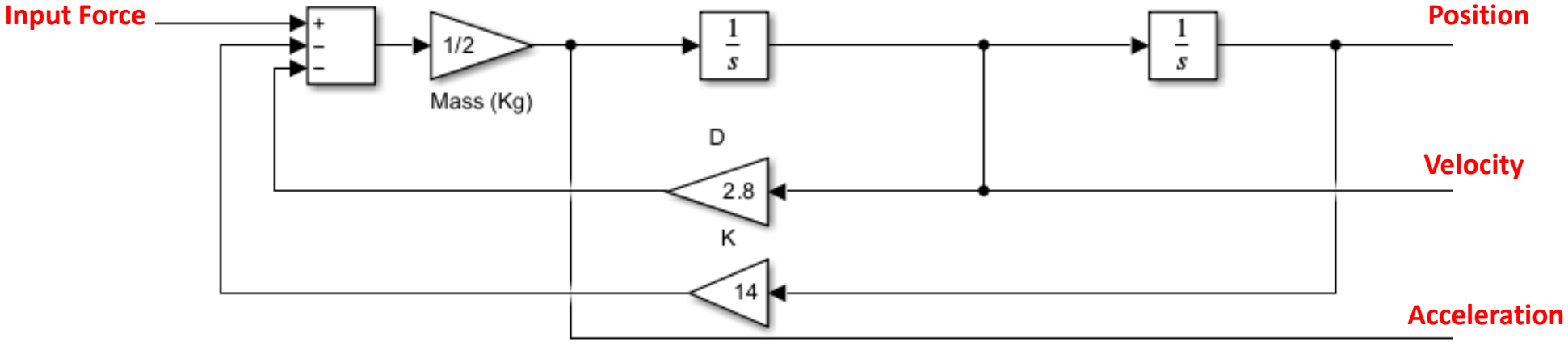
$$F(t) = m\ddot{x}(t) + D\dot{x}(t) + Kx(t)$$

Step 1: Build an 'integrative path':



Step 2: Sum the signals to realize the equation:

Qui ha creato lo schema considerando l'equazione scritta come: F(t)-Dx'(t)-Kx(t)=mx''(t)



Tip: **Put the input force to zero to check the free response of the system**

Blocks required: **Integrator**, **Gain**, **Adder** (or **Sum**). Also use a **Scope** to visualize signals

UNIVERSITÀ DEGLI STUDI DI GENOVA

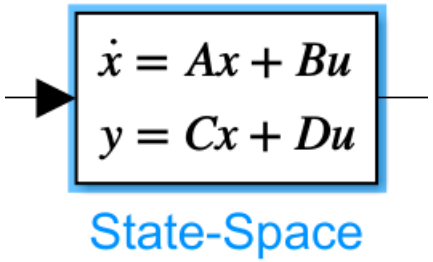
Simulate Differential Equations: State Space

- Diff. Equation: $F(t) = m\ddot{x}(t) + D\dot{x}(t) + Kx(t)$
- State Space model: $\begin{cases} \dot{x} = Ax + Bu \\ y = Cx + Du \end{cases}$
- $x_1 = x; x_2 = \dot{x}; F = u$
- $\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = -\frac{K}{m}x_1 - \frac{D}{m}x_2 + \frac{1}{m}u \end{cases}$
- $\dot{x} = \begin{bmatrix} 0 & 1 \\ -K/m & -D/m \end{bmatrix} x + \begin{bmatrix} 0 \\ 1/m \end{bmatrix} u$

x – state
u – input
y - output

All these are vectors

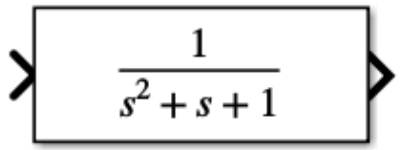
Block:



Simulate Differential Equations: Transfer Function

- Diff. Equation: $\boldsymbol{F(t) = m\ddot{x}(t) + D\dot{x}(t) + Kx(t)}$
- Transfer function (Laplace domain): $\boldsymbol{Y(s) = T(s)U(s);\quad T(s) = \frac{Y(s)}{U(s)}}$
- $\boldsymbol{x(t) \rightarrow Y(s);\ \ \dot{x}(t) \rightarrow sY(s);\ \ \ddot{x}(t) \rightarrow s^2Y(s);\ \ F(t) \rightarrow U(s)}$ – Laplace transform
- $\boldsymbol{U(s) = ms^2Y(s) + DsY(s) + KY(s) = \left(ms^2 + Ds + K\right)Y(s)}$
- $\boldsymbol{T(s) = \frac{Y(s)}{U(s)} = \frac{1}{ms^2+Ds+K}}$

Block:



The PID Controller

Proportional **I**ntegrative **D**erivative signal: $\mathbf{U}(t) = \boldsymbol{K_P e(t)} + \boldsymbol{K_I} \int \boldsymbol{e(t)} + \boldsymbol{K_D \dot{e}(t)}$

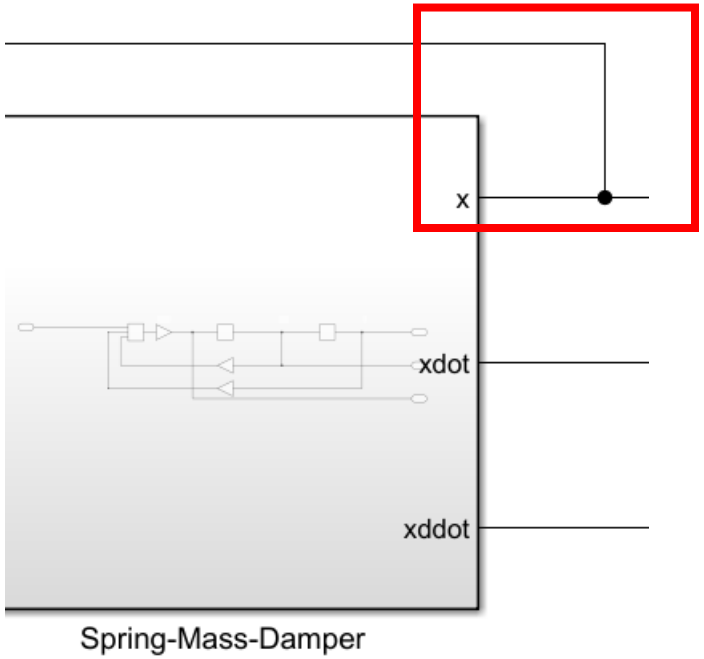
Control Error Tracking

Steady State error zeroing

Transient Response Damping

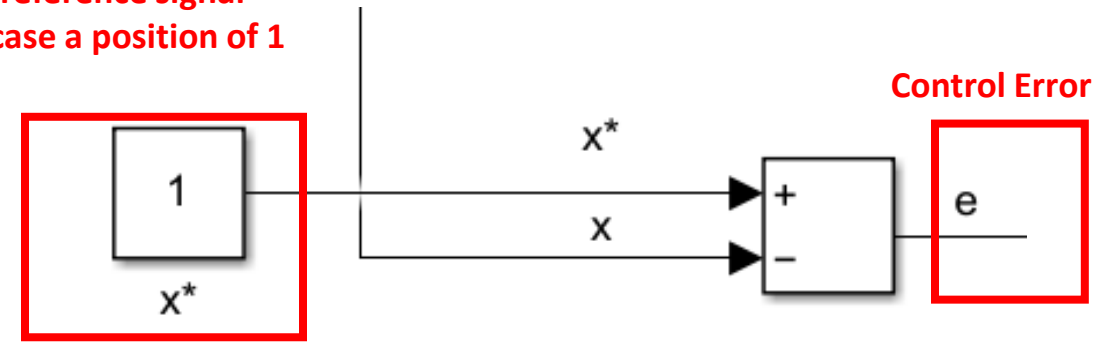
Step 1: Get state measurements and compute the control error

Position Measurement



Desired reference signal (In this case a position of 1 cm)

Control Error



Blocks required: **Integrator**, **Gain**, **Adder** (or **Sum**), **Derivator**

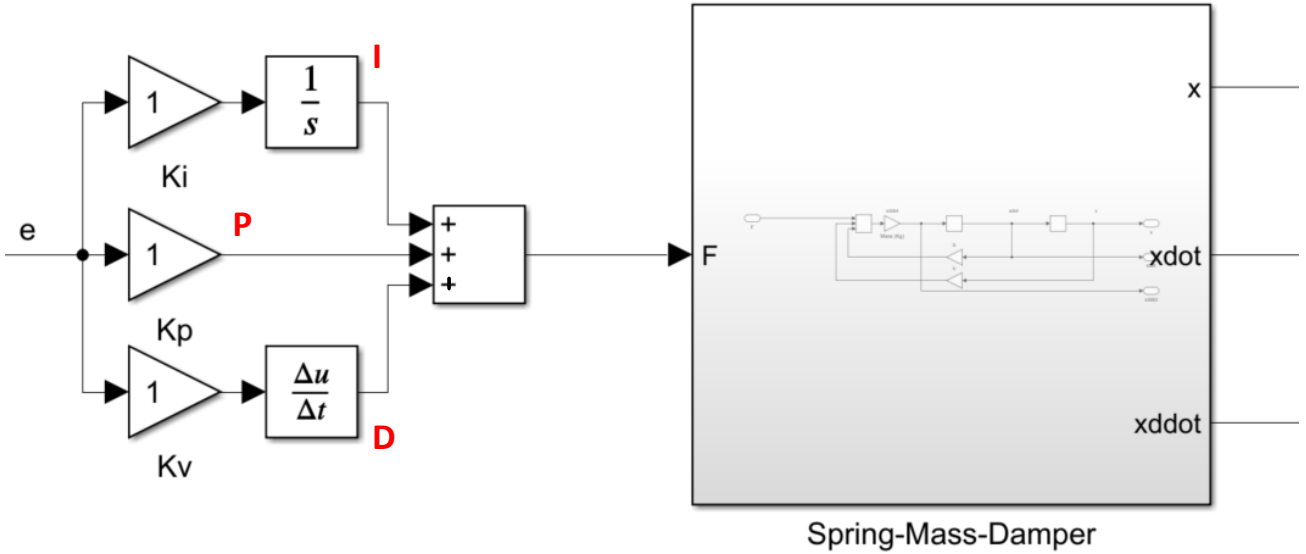
UNIVERSITÀ DEGLI STUDI DI GENOVA

The PID Controller

Proportional **I**ntegrative **D**erivative signal: $\mathbf{U}(t) = \boldsymbol{K_P e}(t) + \boldsymbol{K_I} \int \boldsymbol{e}(t) + \boldsymbol{K_D \dot{e}}(t)$

Control Error Tracking — Steady State error zeroing — Transient Response Damping

Step 2: Build the PID Law and plug the controller ($\mathbf{U}(t) = \mathbf{F}(\mathbf{t})$)



Unit gains won't work well, try some tuning

Tip: **Build the controller incrementally, start with the proportional, then derivative, then integrator and check that each separate component works fine**

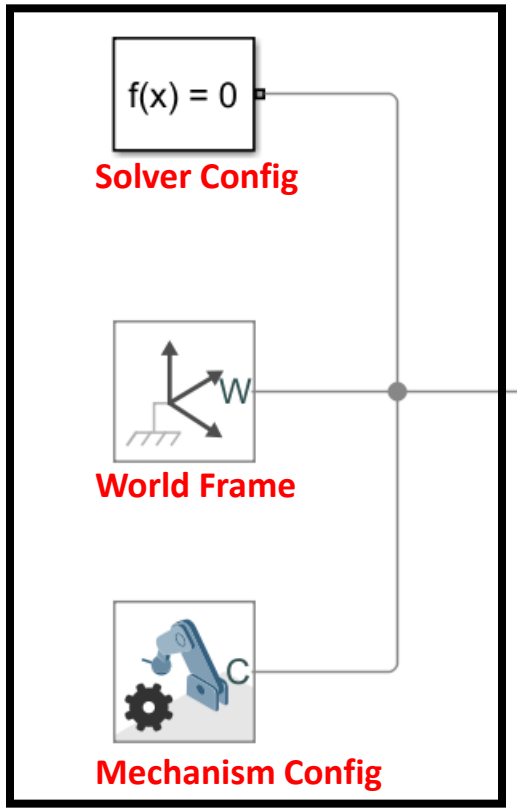
Blocks required: **Integrator**, **Gain**, **Adder** (or **Sum**), **Derivator**

UNIVERSITÀ DEGLI STUDI DI GENOVA

Simscape model

Spring-Mass-Damper is a simple model. **Multi-body systems** (such as robots) can **grow in complexity** as the **number of articulated bodies grows**. You have some alternatives:

- Plain Simulink: **State-Space Representation, Transfer Function**
- Simscape: **Multibody block-modeling**



Solver Config

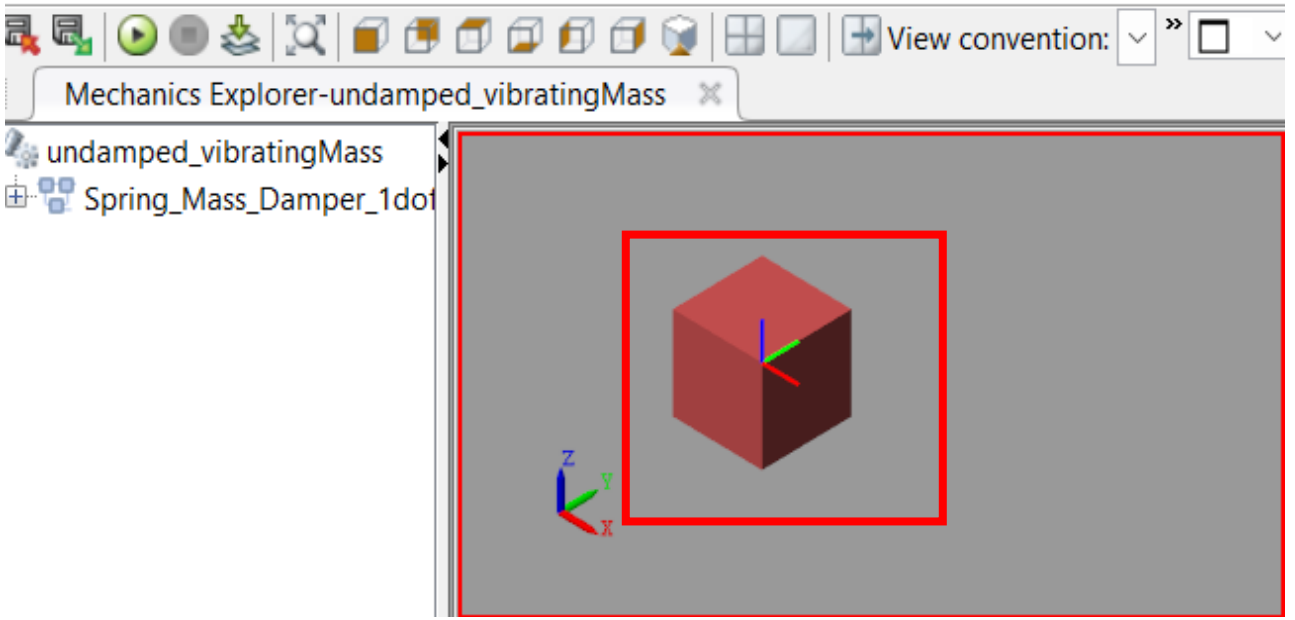
These blocks must be in the model by default!

World Frame

Brick Solid

Add a mass attached to the world frame, by double-click on it you can edit all the visual and mechanical properties

Mechanics Explorer, will toggle each time you run a simulation

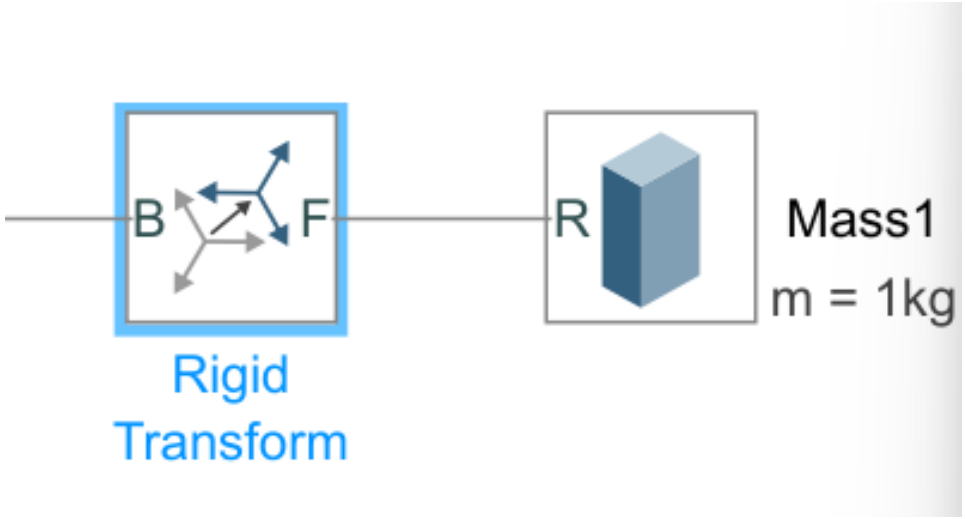


Mass is centered in origin frame, no transformation is applied

UNIVERSITÀ DEGLI STUDI DI GENOVA

Simscape model

You can add *Transformations* between bodies (or a body and the origin frame in this case)



Double click con the transform block to toggle editor

Block Parameters: Rigid Transform

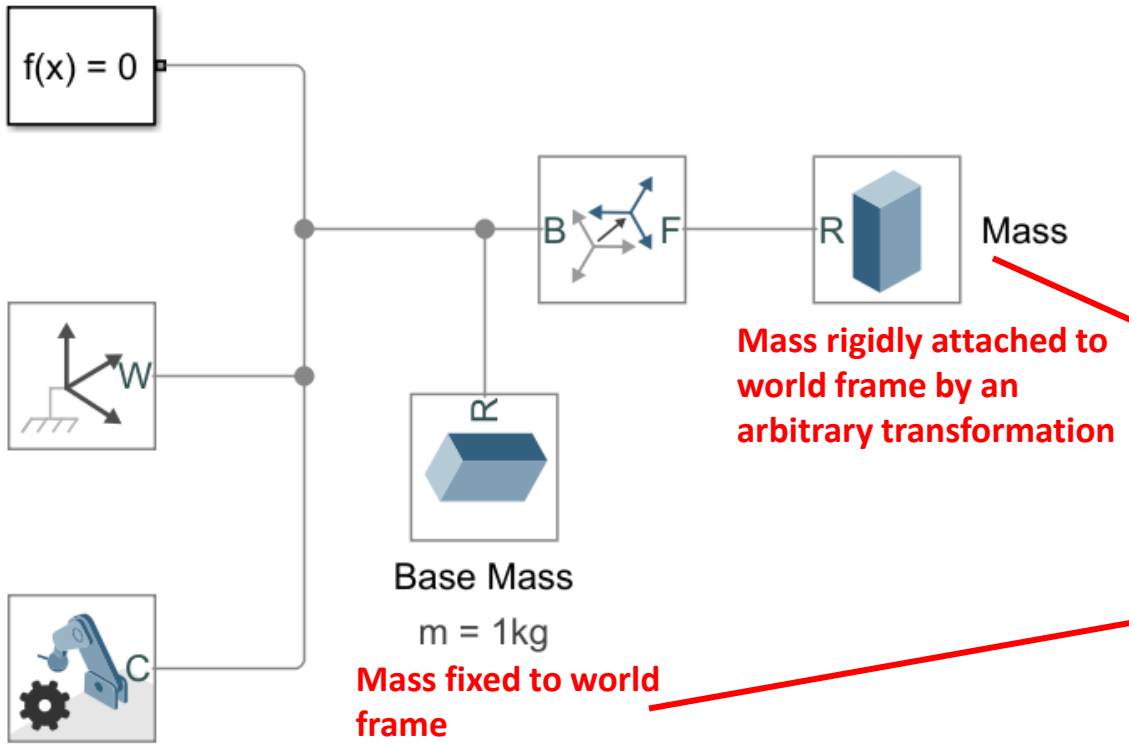
Rigid Transform — Auto Apply

Settings | Description

NAME	VALUE		
Rotation			
Method	Standard Axis		
Axis	+Y		
Angle	90	deg	Compile-time
Translation			
Method	Standard Axis		
Axis	+X		
Offset	15	cm	Compile-time

Simscape model

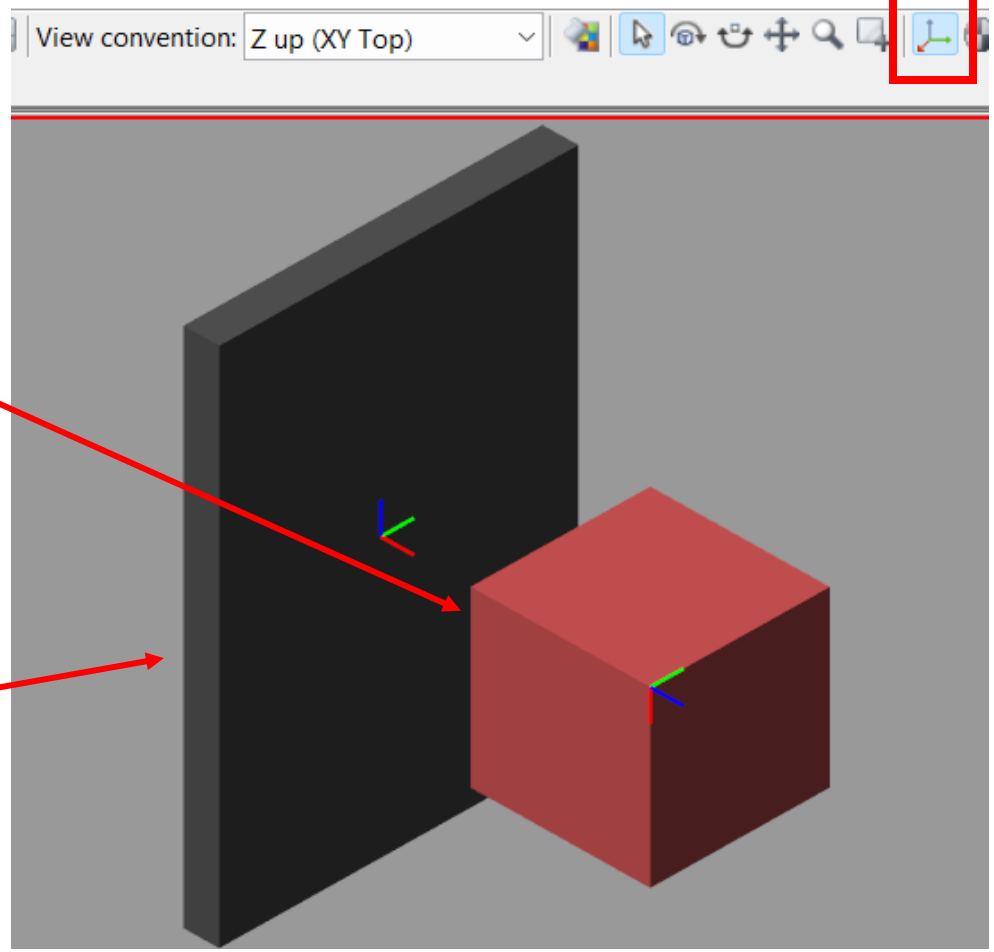
Add another body to the scene, visualize coordinate frames



Mass rigidly attached to world frame by an arbitrary transformation

Mass fixed to world frame

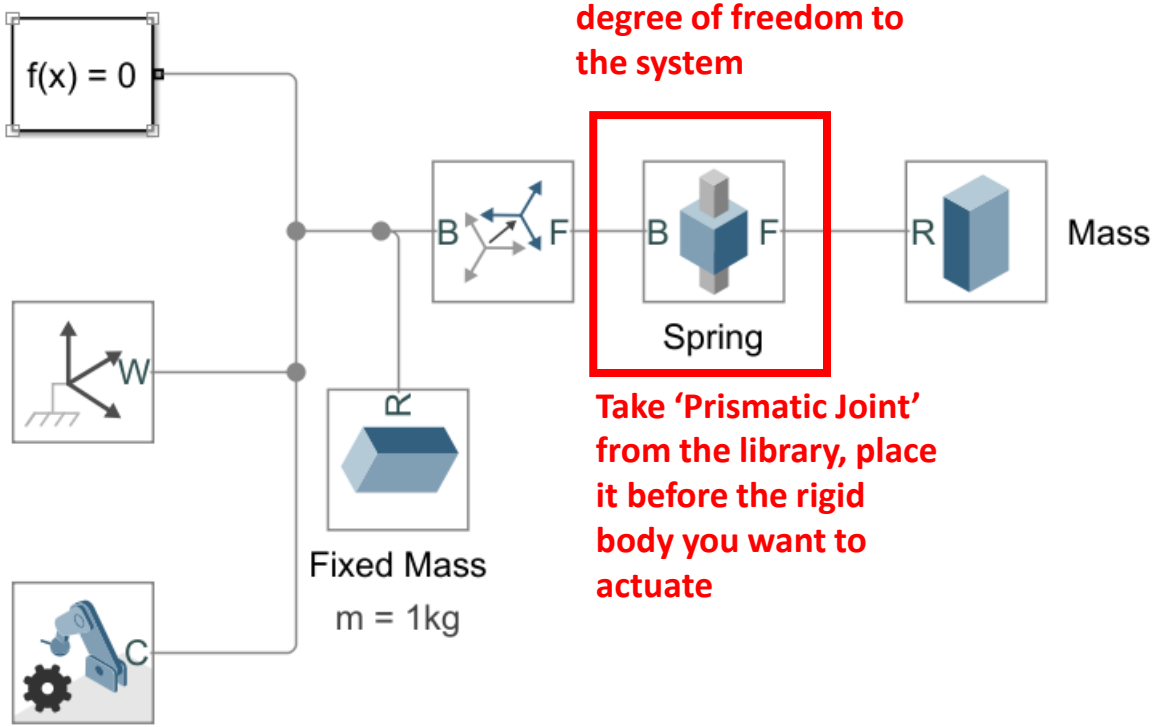
'Toggle frames'



UNIVERSITÀ DEGLI STUDI DI GENOVA

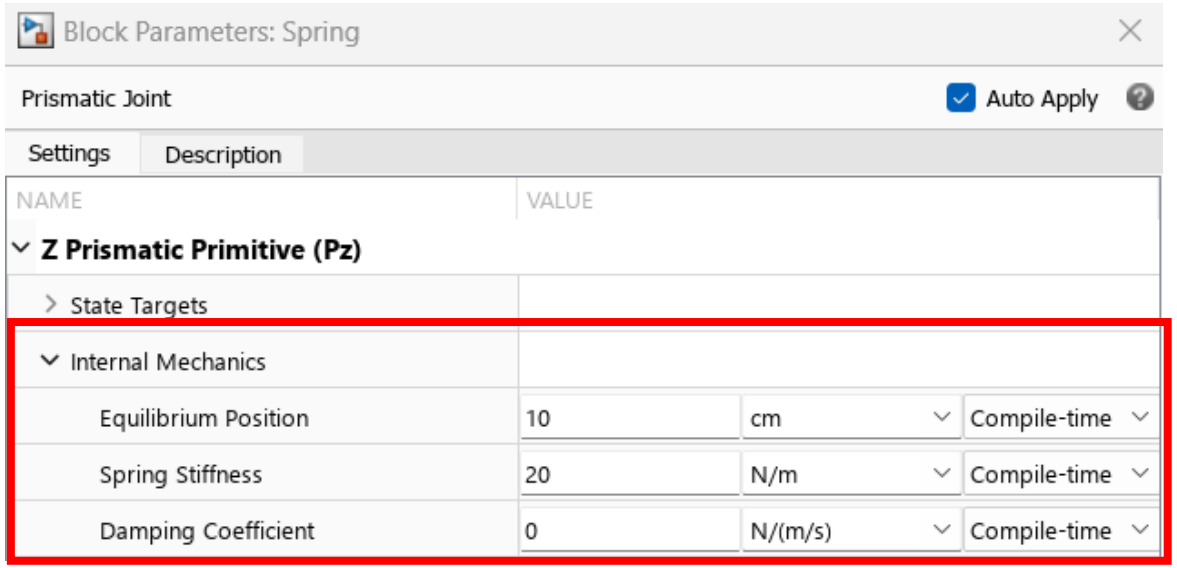
Simscape - Prismatic Joint

Add a prismatic joint to move the mass



This joint adds 1 degree of freedom to the system

Take 'Prismatic Joint' from the library, place it before the rigid body you want to actuate

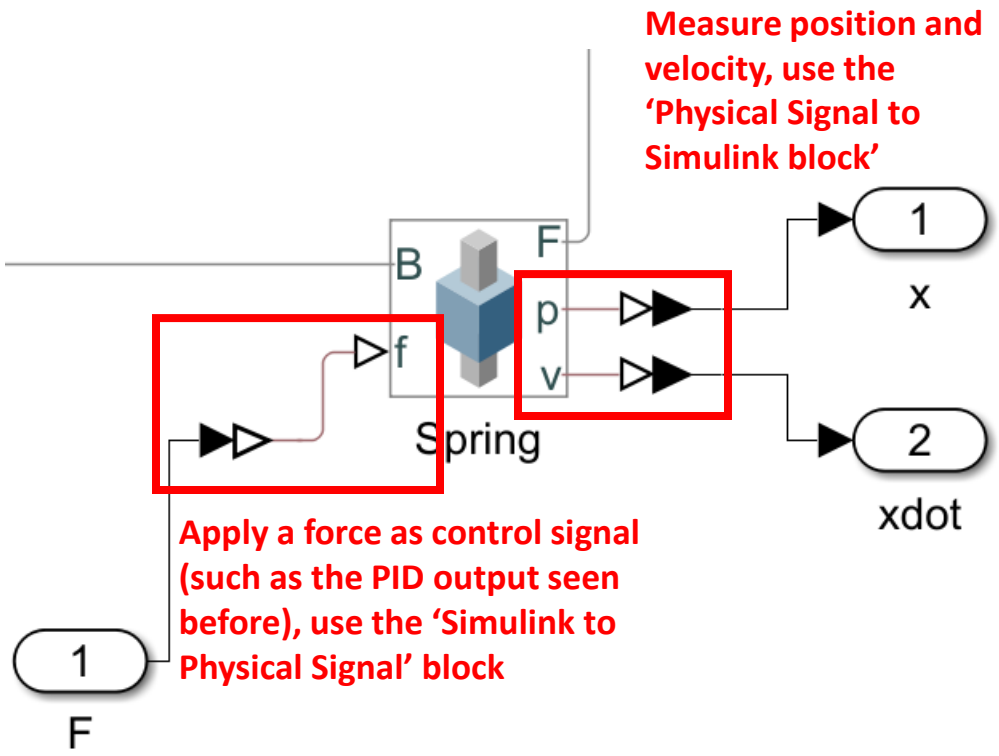


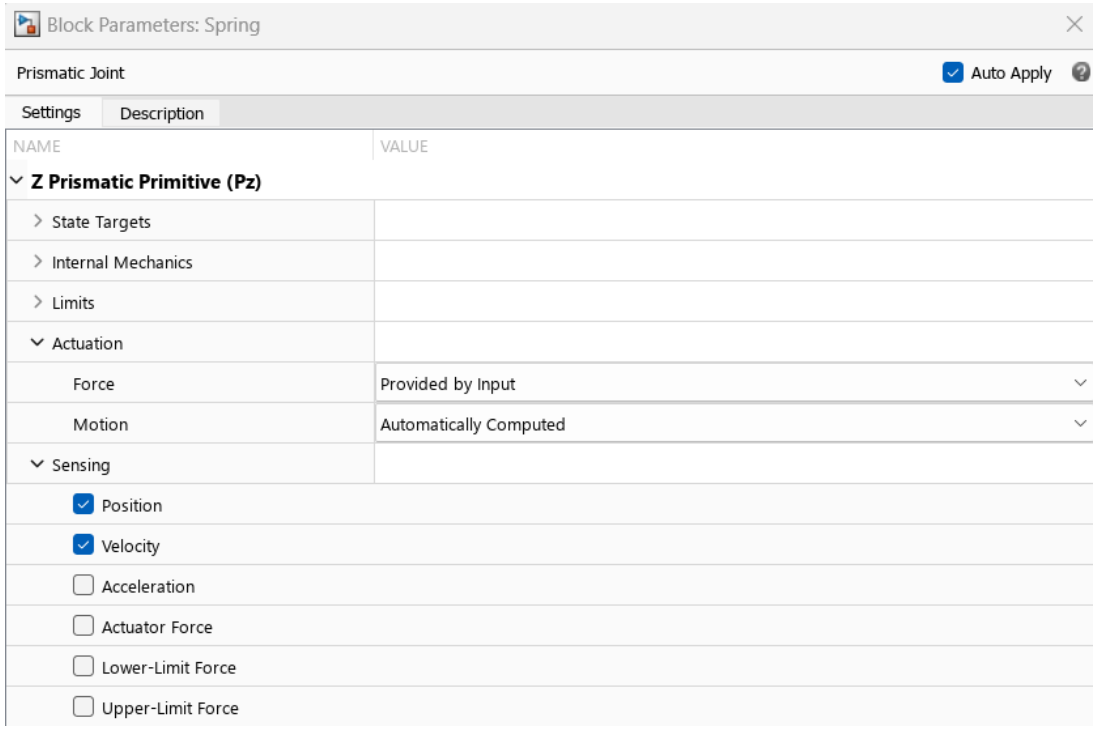
Here you can specify the physical properties of the 'free' (uncontrolled) system behaviour

UNIVERSITÀ DEGLI STUDI DI GENOVA

Linear Actuator Motion Control

Use the prismatic joint to simulate a *linear actuator:* you can add external sensing and actuation



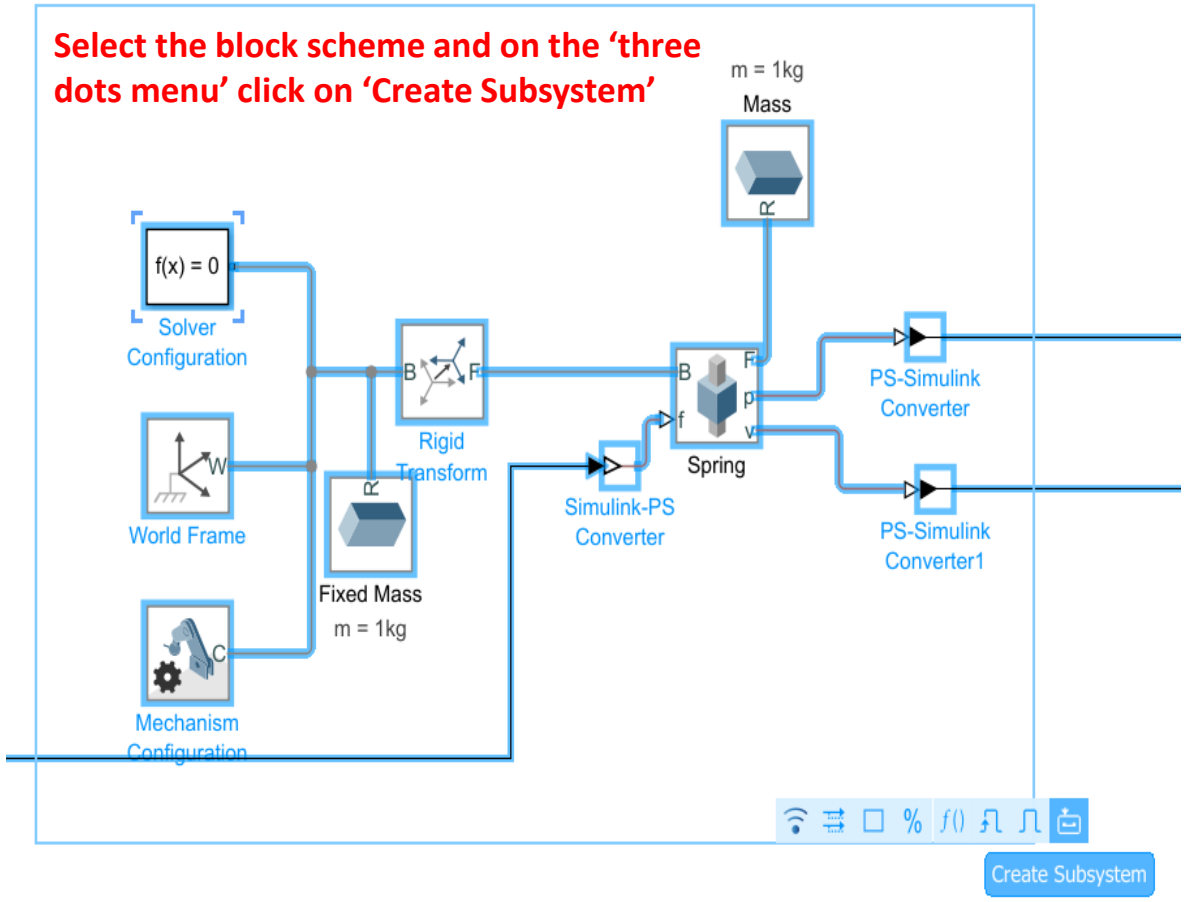


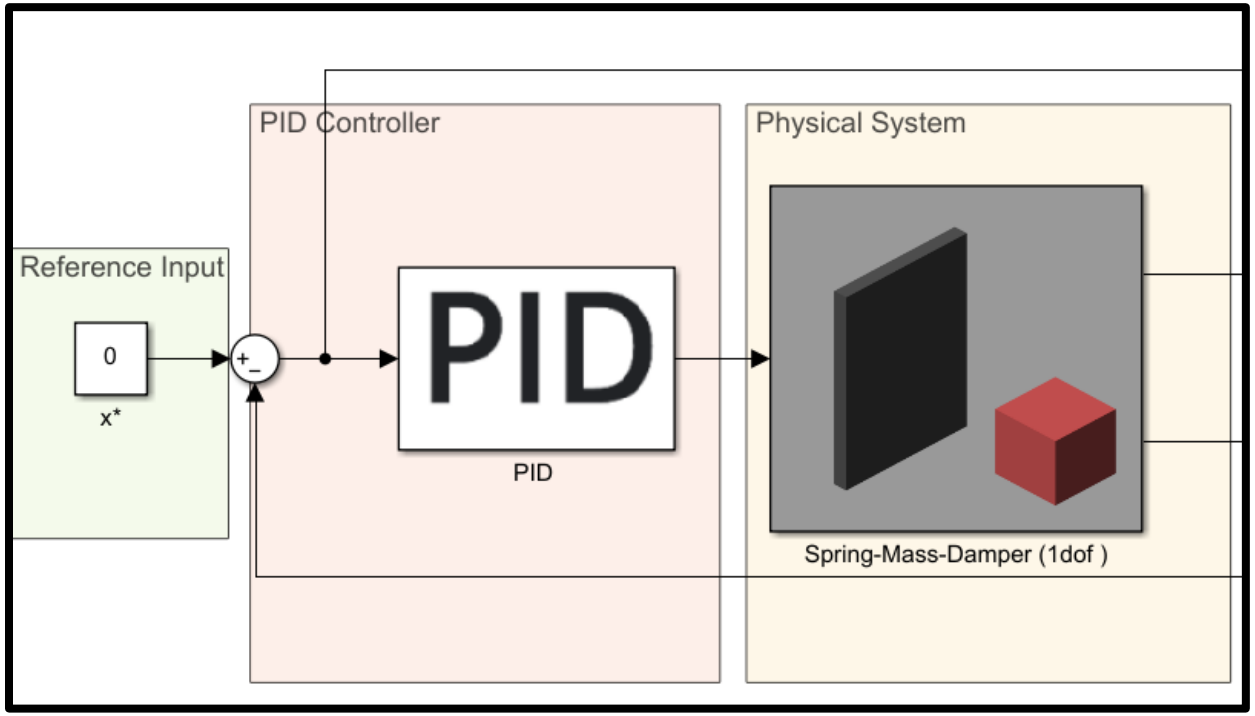
IMPORTANT: The STATE TARGETS are actually the INITIAL CONDITIONS of the joint

UNIVERSITÀ DEGLI STUDI DI GENOVA

Summing up

Make a block which encapsulates the behavior of the physical system, along with sensing and actuation





Then like before you can experiment with control algorithms

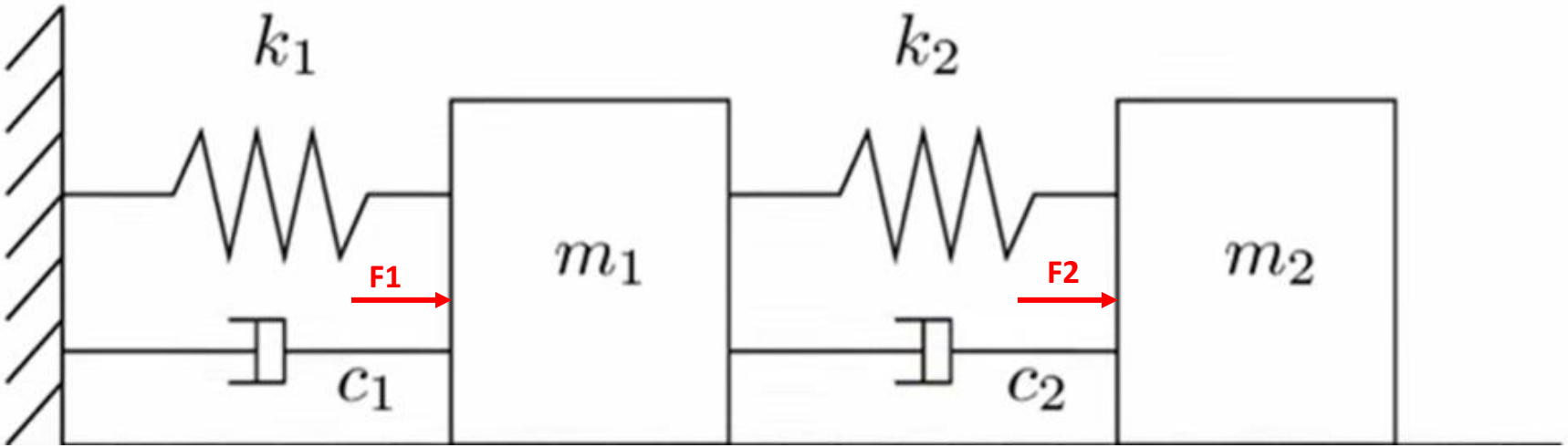
UNIVERSITÀ DEGLI STUDI DI GENOVA

Exercise

D3=D2 e K3=K2

F2(t)-D2(x2'-x1')-K2(x2-x1)=(m2)x2''(t)

F1(t)-D1x1'(t)-k1x1(t)-D3(x1'-x2')-k3(x1-x2)=m1x1''(t)



- Build the **motion equations** using **Newton's Law**
- Build a **Simulink model** of the **system**
- Build a **Simscape Multibody** model
- Test **PID control** on both models. Experiment with inputs (i.e. actuation only on m1, only on m2, both)

'Why do we need this?'

This is not robot modeling yet, but many mechanical models of robot actuators and their controllers are built on top of these principles (you can look for Series Elastic Actuators as an example).
Rotational Joints will be explored in following tutorials.